

Universidade do Minho

Universidade do Minho

Escola de Engenharia

Departamento de Informática

TaxiGreen: Gestão Inteligente de Frota

Inteligência Artificial

Trabalho Prático de Grupo

Ano Letivo 2024/2025

Grupo 18

Francisco Soares - A106882
Carlos Silva - A106792
Nuno Soares - A107366
Carolina Gonçalves - A106897

Janeiro de 2026

Avaliação pelos Pares

Após análise coletiva do contributo de cada elemento do grupo, apresentamos os deltas correspondentes à componente de avaliação pelos pares. Esta avaliação reflete o esforço e dedicação individual de cada membro ao longo do desenvolvimento do trabalho.

Número	**Nome**	**DELTA**
A106882	Francisco Soares	+0.15
A106792	Carlos Silva	+0.15
A107366	Nuno Soares	+0.15
A106897	Carolina Gonçalves	-0.45

Nota: O somatório dos DELTAs é igual a 0.00.

Contents

1 Introdução **3**

2 Descrição do Problema **3**

3 Formulação do Problema **3**

4 Representação da Cidade **4**

5 Algoritmos de Procura **4**
5.1 Algoritmos Não-Informados 4
5.2 Algoritmos Informados 5
5.3 Comparação de Complexidade 5
5.4 Heurísticas Implementadas 5

6 Estratégias de Alocação **5**
6.1 Veículo Mais Próximo (NearestVehicleStrategy) 5
6.2 Menor Custo (LowestCostStrategy) 5
6.3 Ecológica (EcoFriendlyStrategy) 6

7 Simulação Dinâmica **6**

8 Resultados e Avaliação **6**
8.1 Configuração Experimental 6
8.2 Métricas Quantitativas 6
8.3 Análise dos Resultados 6

9 Discussão e Decisões de Design **7**
9.1 Arquitetura Modular 7
9.2 Decisões Técnicas 7
9.3 Tratamento de Casos Limite 7

10 Conclusões **7**

1 Introdução

Neste trabalho, o nosso grupo propôs-se a desenvolver um sistema de gestão inteligente para a frota da TaxiGreen, uma empresa de táxis urbanos que opera com veículos elétricos e a combustão. Reconhecemos que a gestão eficiente de uma frota heterogénea representa um desafio complexo, especialmente quando consideramos a necessidade de equilibrar custos operacionais, tempos de resposta e impacto ambiental.

A TaxiGreen enfrenta desafios crescentes na sua operação diária. Por um lado, os veículos elétricos oferecem custos operacionais reduzidos e zero emissões locais, mas apresentam autonomia limitada e tempos de recarga significativos. Por outro lado, os veículos a combustão possuem maior flexibilidade operacional, embora com custos e impacto ambiental superiores. O nosso sistema pretende encontrar o equilíbrio ideal entre estas características distintas.

O objetivo principal deste trabalho consiste em desenvolver algoritmos de procura que permitam otimizar a gestão da frota, garantindo eficiência operacional, redução de custos energéticos e cumprimento de critérios ambientais. Ao longo deste relatório, apresentamos a formulação do problema como um problema de procura, descrevemos os algoritmos implementados, analisamos as estratégias de alocação desenvolvidas e discutimos os resultados obtidos nas simulações realizadas.

2 Descrição do Problema

A TaxiGreen opera uma frota mista de veículos elétricos e a combustão, cada tipo com características distintas. Os veículos elétricos têm autonomia de 150-300 km, custo de 0.10-0.15 EUR/km, recarga de 30-60 min e zero emissões. Os veículos a combustão possuem autonomia de 500-700 km, custo de 0.20-0.30 EUR/km, reabastecimento de 5-10 min e emissões de 120-180 g CO_2/km.

O problema é caracterizado por pedidos dinâmicos ao longo do dia, condições de trânsito variáveis e eventos imprevistos (estradas cortadas, falhas em estações). Os principais desafios incluem responder rapidamente aos pedidos, gerir veículos heterogéneos, otimizar custos operacionais e reduzir emissões de CO_2.

Os pedidos de transporte especificam localização de origem e destino, número de passageiros, horário pretendido, nível de prioridade (normal, urgente) e preferência ambiental do cliente. Este conjunto de requisitos torna o problema particularmente complexo, exigindo uma abordagem que considere múltiplos critérios de otimização simultaneamente.

3 Formulação do Problema

Representamos o estado através de um tuplo $S = (V, P, C, T)$, onde V é o conjunto de veículos com propriedades como identificador, tipo, localização, autonomia atual/máxima, capacidade e custo por km. P representa os pedidos pendentes com origem, destino, passageiros, horário e prioridade. C descreve o estado da cidade incluindo grafo de localizações, estações e fatores de trânsito. T indica o tempo atual da simulação.

No estado inicial, todos os veículos estão disponíveis com autonomia máxima, posicionados em localizações estratégicas. A cidade opera em condições normais com trânsito a fator 1.0, todas as estradas abertas e estações funcionais.

O objetivo é satisfazer continuamente os pedidos maximizando a taxa de atendimento, minimizando tempo de resposta, custos operacionais, emissões de CO_2 e quilómetros vazios (percorridos sem passageiros).

Os operadores definidos são:

- `ATRIBUIR_VEICULO(v, p)`: associa veículo a pedido (pré-condições: disponível, capacidade, autonomia)
- `DESLOCAR_VEICULO(v, origem, destino)`: move veículo (consome autonomia, tempo)
- `RECARREGAR(v, e)`: repõe autonomia em estação compatível
- `REJEITAR_PEDIDO(p)`: para casos inviáveis
- `AGUARDAR(v, duracao)`: veículo permanece em espera

A função de custo multi-objetivo pondera diferentes critérios: $C_{total} = \alpha_1 \cdot C_{operacional} + \alpha_2 \cdot C_{tempo} + \alpha_3 \cdot C_{ambiental} + \alpha_4 \cdot C_{penalizacoes}$, onde os pesos α são configuráveis conforme as prioridades operacionais.

4 Representação da Cidade

A cidade é representada como grafo direcionado $G = (N, E)$ onde os nós correspondem a localizações relevantes e as arestas representam ligações rodoviárias. Os tipos de nós incluem:

- Zonas de recolha de passageiros distribuídas pela cidade
- Estações de recarga elétrica para veículos elétricos
- Postos de abastecimento para veículos a combustão

Cada aresta possui atributos de distância em quilómetros, tempo base de viagem calculado a partir da velocidade média, fator de trânsito atual (multiplicador do tempo base) e estado da via (aberta ou cortada).

Esta representação permite modelar condições dinâmicas que afetam a operação:

- Congestionamento de trânsito: fatores entre 1.0 (livre) e 3.0 (severo)
- Estradas cortadas: arestas intransitáveis forçando rotas alternativas
- Disponibilidade de estações: simulando avarias ou manutenção

5 Algoritmos de Procura

5.1 Algoritmos Não-Informados

Implementámos três algoritmos de procura não-informada que exploram o espaço de estados sem utilizar informação específica do domínio:

BFS (Breadth-First Search) explora todos os nós a uma determinada profundidade antes de avançar, utilizando uma fila FIFO. É completo e ótimo para custos uniformes, com complexidade temporal e espacial $O(b^d)$ onde b é o fator de ramificação e d a profundidade da solução.

DFS (Depth-First Search) explora cada ramo até à máxima profundidade antes de retroceder, utilizando uma pilha LIFO. Tem menor consumo de memória com complexidade espacial $O(bm)$ onde m é a profundidade máxima, mas não garante soluções ótimas.

UCS (Uniform Cost Search) expande sempre o nó com menor custo acumulado $g(n)$, utilizando uma fila de prioridade. Garante otimalidade independentemente da variação de custos entre ações, sendo adequado quando os custos variam conforme veículo e trânsito.

5.2 Algoritmos Informados

Desenvolvemos algoritmos que utilizam heurísticas para guiar a procura de forma mais eficiente:

A* combina o custo do caminho $g(n)$ com estimativa heurística $h(n)$, expandindo por ordem de $f(n) = g(n) + h(n)$. É completo e ótimo quando a heurística é admissível (nunca sobrestima o custo real). A eficiência depende diretamente da qualidade da heurística.

Greedy Best-First Search expande o nó com menor valor heurístico $h(n)$, ignorando o custo acumulado. É tipicamente mais rápido pois foca-se no objetivo, mas não garante soluções ótimas e pode ficar presa em mínimos locais.

5.3 Comparação de Complexidade

Algoritmo	Completo	Ótimo	Tempo	Espaço
BFS	Sim	Sim*	$O(b^d)$	$O(b^d)$
DFS	Não	Não	$O(b^m)$	$O(bm)$
UCS	Sim	Sim	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$
A*	Sim	Sim**	$O(b^d)$	$O(b^d)$
Greedy	Não	Não	$O(b^m)$	$O(b^m)$

** Para custos uniformes. ** Para heurísticas admissíveis.*

5.4 Heurísticas Implementadas

Implementámos quatro heurísticas para utilização com os algoritmos informados:

Euclidiana: $h = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ — distância em linha reta, admissível em redes rodoviárias.

Manhattan: $h = |x_2 - x_1| + |y_2 - y_1|$ — soma das diferenças absolutas, útil em grelhas urbanas regulares.

Tempo Estimado: $h = d_{euclidiana}/v_{media}$ — fornece estimativa temporal relevante quando o objetivo é minimizar tempo.

Composta: $h = (d_{euclidiana}/v_{media}) \times f_{transito}$ — ajusta pelo fator de trânsito atual, oferecendo maior realismo.

6 Estratégias de Alocação

Desenvolvemos três estratégias de alocação de veículos a pedidos, cada uma otimizando diferentes critérios:

6.1 Veículo Mais Próximo (NearestVehicleStrategy)

Atribui a cada pedido o veículo disponível com menor distância ao ponto de recolha. Simples e eficiente computacionalmente, minimiza tempo de resposta inicial. Não considera custos da viagem completa nem características do veículo.

6.2 Menor Custo (LowestCostStrategy)

Seleciona o veículo que minimiza o custo operacional total: $(d_{pickup} + d_{viagem}) \times custo_{km}$. Favorece naturalmente veículos elétricos (0.12 EUR/km) face aos de combustão (0.25 EUR/km). Adequada quando o objetivo principal é otimização financeira.

6.3 Ecológica (EcoFriendlyStrategy)

Prioriza veículos elétricos sempre que tenham autonomia suficiente. Primeiro filtra elétricos disponíveis com autonomia adequada e ordena por distância; se nenhum viável, considera combustão. Minimiza emissões de CO_2, alinhando-se com objetivos de sustentabilidade.

7 Simulação Dinâmica

Implementámos um simulador orientado a eventos através da classe `TaxiSimulator`. Esta arquitetura processa ocorrências por ordem temporal, sendo mais eficiente que passos temporais fixos pois apenas processa eventos quando efetivamente ocorrem.

O simulador mantém uma fila de eventos ordenada por tempo. Os tipos de eventos suportados incluem:

- Chegada de novos pedidos
- Atribuição de veículos a pedidos
- Início e fim de viagens
- Início e fim de recargas
- Eventos dinâmicos (trânsito, estradas cortadas, falhas em estações)

O ciclo principal extrai o próximo evento, avança o tempo simulado, processa o evento invocando o handler apropriado e adiciona novos eventos gerados. O ciclo termina quando a fila fica vazia ou se atinge o tempo máximo.

Para simular condições realistas, implementámos mecanismos de atualização de trânsito (fatores multiplicativos nas arestas), bloqueio de estradas (arestas intransitáveis temporariamente) e falhas de estações (indisponibilidade de pontos de recarga).

8 Resultados e Avaliação

8.1 Configuração Experimental

Configurámos uma frota com 5 veículos: 3 elétricos (EV001, EV002, EV003) com autonomia de 250 km e custo de 0.12 EUR/km, e 2 a combustão (CV001, CV002) com 600 km e 0.25 EUR/km. Simulámos 30 pedidos em 180 minutos com semente aleatória fixa para reprodutibilidade.

8.2 Métricas Quantitativas

Os resultados nas seis métricas obrigatórias do enunciado:

Métrica	**Valor**	**Unidade**	**Observação**
Tempo médio resposta	0.14	min	Atribuição rápida
Custos operacionais	32.96	EUR	Predominância elétricos
Emissões CO_2	11.92	kg	Apenas combustão
Taxa atendimento	96.67	%	29/30 pedidos
Km vazios	71.61	km	38% do total

8.3 Análise dos Resultados

Os veículos elétricos apresentaram custo por pedido de 0.55-0.88 EUR versus 1.75-1.87 EUR dos de combustão — diferença de aproximadamente 3x que confirma a vantagem económica quando a autonomia não é limitante. 100% das emissões de CO_2 provêm dos veículos a combustão.

A distribuição foi assimétrica: elétricos atenderam 20 pedidos (7+5+8), combustão atenderam 9 pedidos (4+5), resultado da estratégia que prioriza menor custo. A taxa de 96.67% indica resposta adequada à procura, com apenas 1 pedido pendente por limitações temporais.

Os 38% de km vazios representam oportunidade de melhoria através de posicionamento proativo da frota em zonas de elevada procura.

9 Discussão e Decisões de Design

9.1 Arquitetura Modular

O sistema foi estruturado em cinco módulos independentes:

- `models`: classes de domínio (Vehicle, Request, City, Station)
- `algorithms`: algoritmos de procura e heurísticas
- `strategies`: estratégias de alocação
- `simulation`: simulador orientado a eventos e métricas
- `utils`: classes auxiliares (Node, Problem)

Esta separação facilita testabilidade (cada módulo verificável isoladamente) e extensibilidade (adicionar novos algoritmos ou estratégias sem modificar código existente), seguindo o princípio Open-Closed.

9.2 Decisões Técnicas

Optámos por simulação orientada a eventos por ser mais eficiente que passos temporais fixos, dado que o ambiente apresenta períodos sem atividade significativa. A função de custo multi-objetivo com pesos configuráveis permite ajustar prioridades conforme objetivos operacionais.

9.3 Tratamento de Casos Limite

Durante o desenvolvimento, identificámos e tratámos casos limite:

- Autonomia insuficiente: procura rota via estação de recarga
- Estações indisponíveis: pedidos entram em fila com prioridade
- Empate de custos: distância como critério de desempate

10 Conclusões

O sistema desenvolvido responde aos requisitos do enunciado: 5 algoritmos de procura (BFS, DFS, UCS, A*, Greedy), 4 heurísticas (Euclidiana, Manhattan, Tempo Estimado, Composta), 3 estratégias de alocação (NearestVehicle, LowestCost, EcoFriendly) e 6 métricas obrigatórias.

As simulações demonstraram taxa de atendimento de 96.67% com custos de 32.96 EUR e emissões de 11.92 kg CO_2. O simulador orientado a eventos modelou adequadamente condições dinâmicas. Como trabalho futuro: posicionamento proativo de veículos, previsão de procura e ride-sharing.